

课程笔记

一个 Token 的旅程

2026 南京大学《操作系统原理》第 21 讲

lecture-to-notes

2026 年 5 月 19 日

2026 南京大学
操作系统原理 21

一个 Token 的旅程

Attention is All You Need

每一层 Transformer (head) 都是“一本书”

- 每一本书都输出一个“改写后的版本”（去掉噪声，越来越接近天书）
- 看书：Q: 需要完全读吗？（我在找...） K: 书的目录（我有什么），V: 书的内容（书里面有什么... 甚至这书是学出来的）
- “看书”以后生成“短时记忆”，不需要再看 context，直接经过全连接层“重写”记忆

Diagram illustrating the flow of an input token through a Transformer layer, showing the calculation of weighted output using attention mechanisms.

视频作者/频道：绿导师原谅你了（蒋炎岩 / jyy）

发布日期：2026-05-14

视频时长：1:30:45

视频链接：<https://www.bilibili.com/video/BV1zS5R6tEWZ/>

目录

1 本讲定位：把前 20 讲串成一个故事	2
1.1 先回顾：并发部分讲了什么	2
1.2 本章小结	3
2 数据中心：互联网 & AI 时代的幕后英雄	4
2.1 数据中心的两个时代	4
2.2 C10K 问题：一切并发架构的起点	5
2.3 操作系统课的所有知识在数据中心的落地	5
2.4 本章小结	6
3 从 Token 到 Tensor：LLM 推理的硬件视角	7
3.1 LLM 推理 = 一个巨大的数学函数采样	7
3.1.1 GPT-2 forward pass 的真容	7
3.2 Tensor：广义化的数组	8
3.3 Attention is All You Need：用 SIMT 实现	8
3.4 Token 旅程的最后一公里	9
3.5 本章小结	9
4 总结与延伸	10
4.1 讲者核心观点	10
4.2 自测三问	10
4.3 与课程其他部分的 synthesis	10
4.4 拓展阅读	11
4.5 致谢	11

1 本讲定位：把前 20 讲串成一个故事

这是并发部分的收官，也是整门课的一个高潮——**用一个具体场景把前面所有知识串起来**：当你在终端里给 LLM 发出一句话、按下回车，从这一刻到屏幕上吐出第一个 token 之间，发生了什么？

Token 的旅程，贯穿全部 20 讲

- 你的进程 → 系统调用 → 网络栈（讲 5-7, 12）
- DNS 解析 → TCP 三次握手 → TLS（讲 19 的事件循环）
- 数据中心 → 负载均衡 → 应用服务器（C10K → C10M, 讲 18 + 19）
- 推理后端的请求队列 → GPU SM 调度（讲 14-17 同步原语）
- Transformer forward pass → CUDA kernel → tensor core（讲 20 SIMT）
- Token 通过 SSE / WebSocket 流式返回你的终端（讲 19 异步）

操作系统课的每一块拼图，都是这趟旅程的一个驿站——本讲把它们连起来。

1.1 先回顾：并发部分讲了什么

5 周的并发部分有几条主线：

- **Spawn / Join** 打开了魔鬼的盒子（讲 13）；
- **Mutex / 条件变量 / 信号量 / RCU** 关上盒子（讲 14-17）；
- **并行算法 + 并行数据结构** 让 scale up / scale out 成为可能（讲 18）；
- **协程 / Goroutine / async-await** 让”线程”变得便宜（讲 19）；
- **SIMD / SIMT / GPU** 把并发推到硬件极限（讲 20）；
- **本讲（21）** = 用一个端到端故事把这些机制全部”激活”。

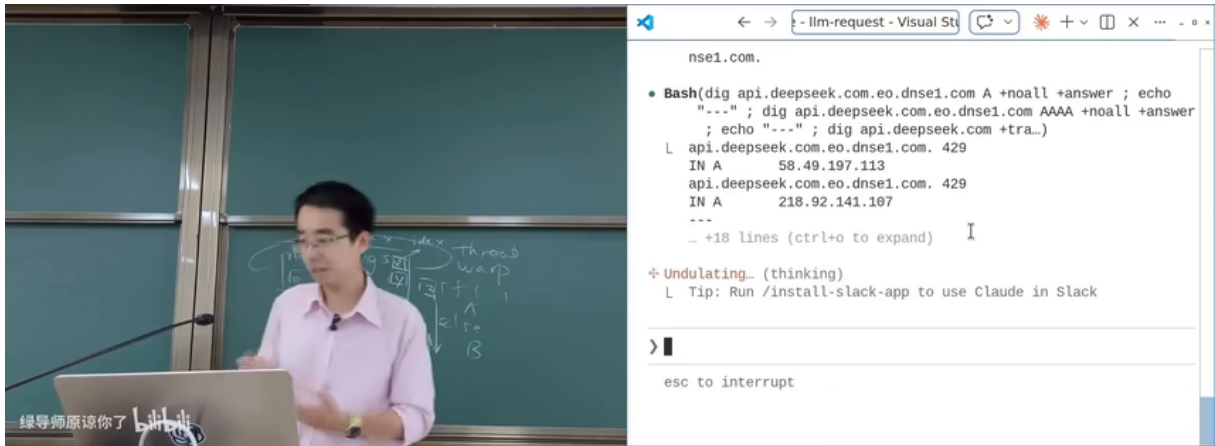


图 1: Token 旅程的起点: 在 Claude Code CLI 里执行 `dig api.deepseek.com.eo.dnse1.com A +noall +answer`——你看到的”API endpoint”其实是一个被 DNS 多次 CNAME 重定向的入口, 最终落到 58.49.197.113 这样的物理 IP。那个 IP 背后是一栋数据中心 + 一片 GPU 集群¹

1.2 本章小结

每一节并发课的知识, 最后都在”你按下回车 → 屏幕吐 token”这个 60 秒的过程里发挥作用。后面两章会拆开这个过程: 先讲数据中心怎么处理你这个请求, 再讲 token 是怎么从一堆参数里被算出来的。

¹ 视频画面时间区间: 00:13:30–00:13:45。

2 数据中心：互联网 & AI 时代的幕后英雄

2.1 数据中心的两个时代

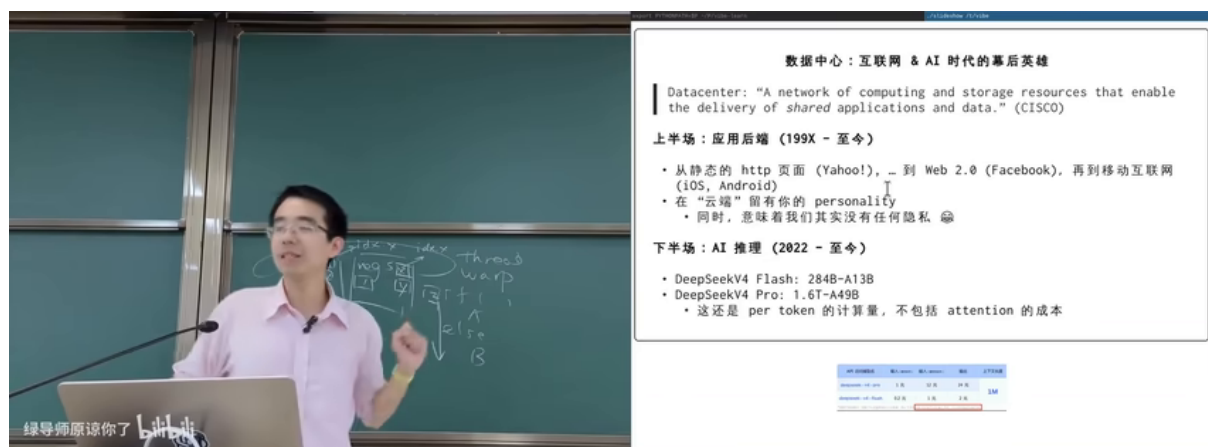


图 2: Datacenter = "A network of computing and storage resources that enable the delivery of shared applications and data" (CISCO). **上半场 (199X-至今)：应用后端**——Yahoo! → Web 2.0 (Facebook) → 移动互联网 (iOS/Android)；你的所有数据都在云端，"personality" 也在云端——也意味着我们其实没有任何隐私。**下半场 (2022-至今)：AI 推理**——DeepSeek V4 Flash 284B-A13B、Pro 1.6T-A49B（数字是 每个 *token* 的激活参数量，不含 attention 成本）²

为什么数据中心是一个产业

当你的服务要同时服务全世界几十亿人，"做一道 OJ 题" 那种思维就完全不够用了。数据中心需要**土地 + 水 + 电 + 网 + 安保 + 人**——这是一条非常长的产业链。今天最贵的"算力" 几乎全部为 AI training/inference 服务：升腾 950 上线、DeepSeek 再次降价、XAI 解散转给 Anthropic——**算力供给和模型迭代**已经成了驱动整个 IT 产业的核心引擎。

²视频画面时间区间：00:22:30-00:22:45。

2.2 C10K 问题：一切并发架构的起点

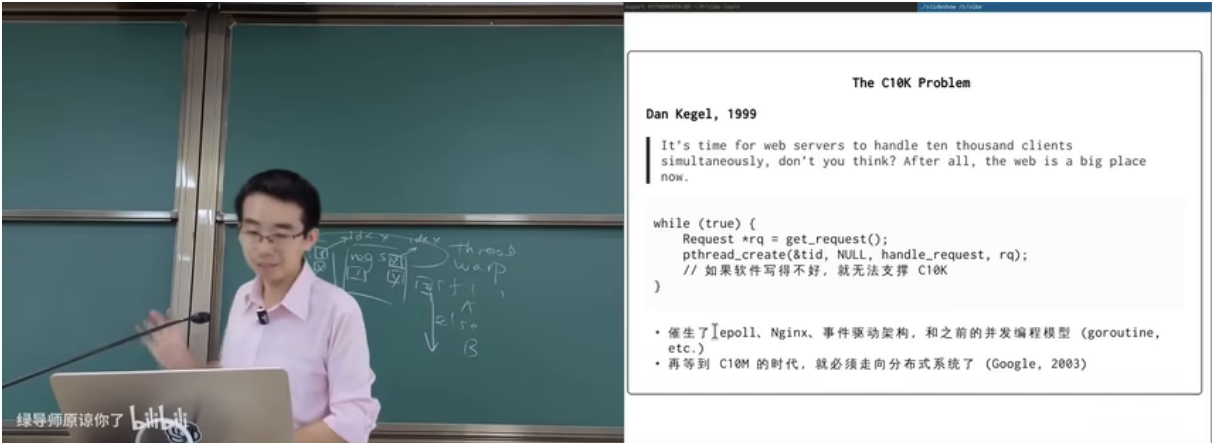


图 3: The C10K Problem (Dan Kegel, 1999): "It's time for web servers to handle ten thousand clients simultaneously." 朴素的 `while(true) { pthread_create(handle_request); }` 写法 无法支撑 C10K——10000 个 OS 线程 = 80 GB 栈 + 上下文切换雪崩。催生了 `epoll`、`Nginx`、事件驱动架构、协程编程模型 (`goroutine`...); 更进一步的 C10M 时代 (Google 2003) 则不可避免地走向了分布式系统³

C10K → C10M → C10B 演化轨迹

- C10K (1999): 单机内并发模型革命——`epoll`、`Nginx`、`Node.js`;
- C10M (2010s): 单机 1000 万连接——零拷贝、用户态网络栈、`DPDK`;
- C10B (2020s): 百亿并发——分布式 `service mesh`、`CDN` + 边缘计算、智能负载均衡;
- AI 推理时代: 每个连接背后是一次 `GPU` 推理——并发量不再决定一切, 每请求的算力消耗成为新瓶颈。

2.3 操作系统课的所有知识在数据中心的落地

一个 HTTP 请求穿过数据中心的全程:

1	client browser	
2	DNS query (UDP)	<- 讲 7 socket fd
3	TCP handshake (SYN/SYN-ACK/ACK)	<- 讲 14 mutex (kernel)
4	TLS handshake	<- 讲 13 加密线程池
5	v	
6	CDN edge node (Cloudflare/Akamai)	
7	Route to nearest datacenter	<- 讲 18 并行算法 (consistent hash)
8	v	
9	L4 load balancer (XDP/eBPF)	
10	DNAT to a healthy backend	<- 讲 17 内核并发数据结构
11	v	
12	L7 reverse proxy (Nginx/Envoy)	
13	parse HTTP, request routing	<- 讲 19 协程/epoll

³视频画面时间区间: 00:33:30-00:33:45。

```

14     v
15 application server (Go/Rust/Python)
16   | enqueue inference request          <- 讲 14-15 condvar + queue
17     v
18 GPU scheduler
19   | pick a free GPU, dispatch kernel   <- 讲 18 scheduling
20     v
21 CUDA stream
22   | matmul + attention + ...           <- 讲 20 SIMT
23     v
24 token stream (SSE / WebSocket)
25   | streamed back to client            <- 讲 19 async event loop
26     v
27 client browser displays token

```

Listing 1: Token 旅程的”操作系统视角”全过程

每一步用到的就是前 20 讲的某个具体机制——这正是讲者说”今天没有新概念，就是把所有东西串起来”的含义。

2.4 本章小结

数据中心是所有操作系统知识在工业级规模上的落地战场：网络栈、并发模型、负载均衡、缓存、调度——每一项都源于本课程的某节课。AI 推理时代让这套机器学着适应一个全新瓶颈：每请求的 GPU 算力供给。

3 从 Token 到 Tensor: LLM 推理的硬件视角

3.1 LLM 推理 = 一个巨大的数学函数采样

大模型推理本质上是从一个学到的概率分布里采样下一个 token:

$$p(\text{token}_{t+1} \mid \text{token}_{1:t}) = f_{\theta}(\text{token}_{1:t})$$

- θ : 模型参数 (GPT-2 是 124M、DeepSeek V4 Pro 是 1.6T);
- f_{θ} : Transformer 网络——堆叠的 **Attention + MLP layer**;
- 输出是一个长度 = vocab_size (约 5 万) 的概率分布。

3.1.1 GPT-2 forward pass 的真容

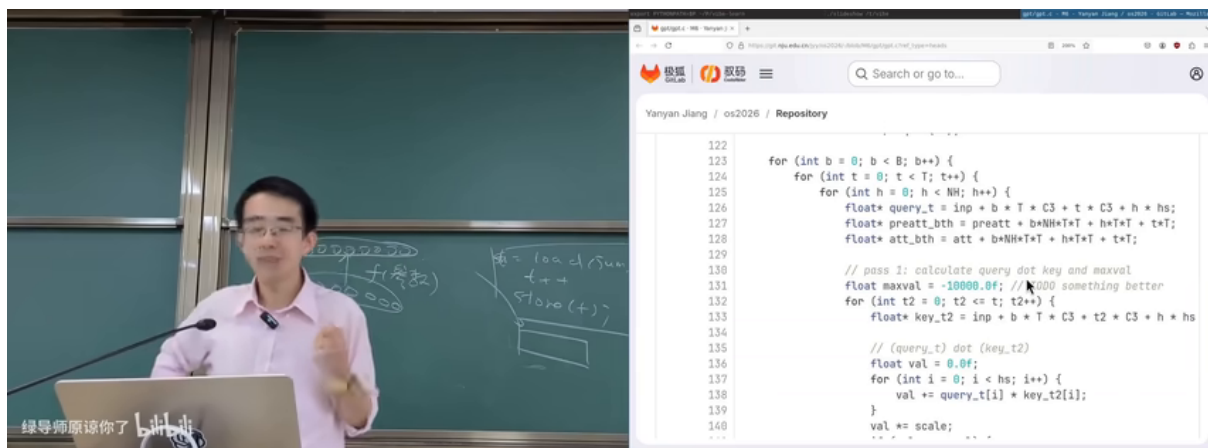


图 4: gpt2.c forward pass: for (b = 0; b < B; b++) for (t = 0; t < T; t++) for (h = 0; h < NH; h++) {...} ——三重循环遍历 batch、time step、head; query_t/preatt_bth/att_bth 通过线性偏移寻址 (b*T*C3 + t*C3 + h*hs) ——这只是 Attention 的一部分实现。”看起来很 fancy 的 transformer 其实就是一大堆数字 + 一大堆算这些数字位置的下标”⁴

Karpathy 的 llm.c: 把神话拉下神坛

Andrej Karpathy 的 llm.c 项目用 ~2000 行纯 C 代码实现了 GPT-2 的训练 + 推理——no PyTorch, no TensorFlow。代码里能看到的只有:

- 几千行 for 循环 (matmul、attention、LayerNorm、softmax);
- 一个 model_t struct 装所有参数;
- 一些线程并行化的 #pragma omp parallel for。

看完一遍你会意识到: ”大模型”在原理层面完全可以理解——它就是一个巨大的数学函数 + 一组巨大的常数。神秘感全部来自工程上的规模。

⁴视频画面时间区间: 01:03:00-01:03:15。

3.2 Tensor: 广义化的数组

- **Scalar** (标量, 0 维): 一个数, 如 3.14;
- **Vector** (向量, 1 维): 一行数组;
- **Matrix** (矩阵, 2 维): 二维数组;
- **Tensor** (张量, N 维): N 维数组。

为什么需要张量? 考虑一张彩色图: $[H, W, C]$ (高 $\times$ 宽 $\times$ 通道, RGB 是 3 个通道); 考虑一个 batch 的彩色图: $[B, H, W, C]$ (4 维); 考虑一个视频 batch: $[B, T, H, W, C]$ (5 维); 考虑 attention 张量 $[B, H, T, T]$ (batch $\times$ head $\times$ seq_len $\times$ seq_len)。

AlexNet (2012): 深度学习重新崛起的爆点

Krizhevsky / Sutskever / Hinton 用 **2 张 GTX 580 GPU** (共 3 GB 显存!) 训练了一个 60M 参数的 CNN, 在 ImageNet 比赛上一举刷新历史记录。这是**把矩阵运算放到 GPU 上跑**的标志性成功——之前大家想到 GPU 只想到游戏, AlexNet 之后所有深度学习都迁移到了 GPU。当时还没有 PyTorch / TensorFlow——他们用 CUDA C++ 手写了所有 kernel。

3.3 Attention is All You Need: 用 SIMT 实现

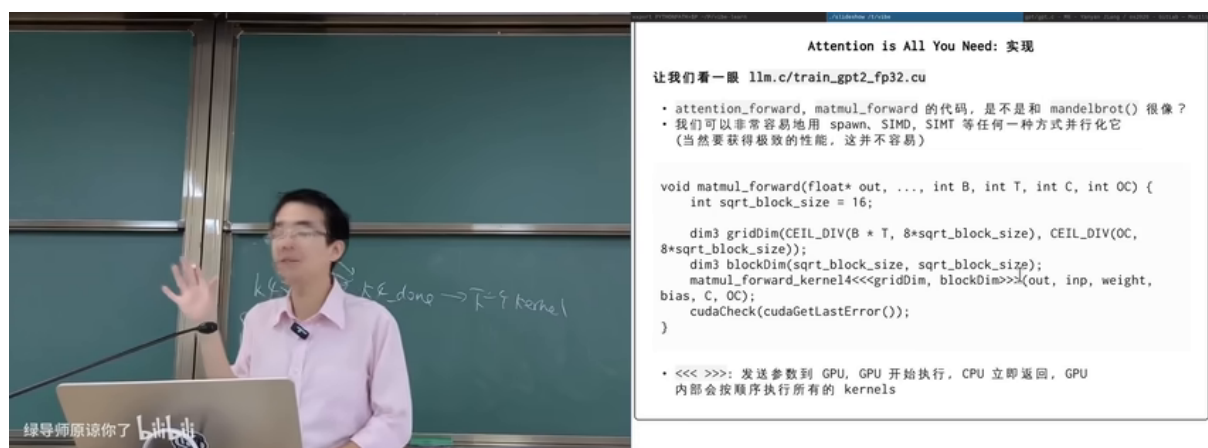


图 5: `llm.c/train_gpt2_fp32.cu` 的 `matmul_forward_kernel`: `dim3 gridDim(CEIL_DIV(B*T, 8*sqrt_block_size), CEIL_DIV(OC, 8*sqrt_block_size)); dim3 blockDim(sqrt_block_size, sqrt_block_size); matmul_forward_kernel<<<gridDim, blockDim>>>(out, inp, weight, bias, ...)`. “<<< >>>: 发送参数到 GPU, GPU 开始执行, CPU 立刻返回, GPU 内部会按序执行所有 kernels”——这就是 attention 的全部硬件实现⁵

⁵ 视频画面时间区间: 01:15:00–01:15:15。

为什么 LLM 推理天生适合 SIMT

看 gpt2.c 的循环结构：

```
1 for (b = 0; b < B; b++)          // batch
2     for (t = 0; t < T; t++)      // time step
3         for (h = 0; h < NH; h++) // head
4             // 计算 attention[b][t][h]
```

Listing 2: attention 的循环骨架

每个 (b, t, h) 三元组的计算是完全独立的——正是 SIMT 模型梦寐以求的输入！把 $b * T * NH$ 维度铺平 → 分给几千个 CUDA thread → 每个 thread 算一个 (b, t, h) 。这就是 $\llcorner\llcorner\llcorner\llcorner$ 的“形状”——把独立的循环维度映射到 GPU thread 网格上。

3.4 Token 旅程的最后一公里

GPU 算完 forward pass 后，得到一个 [vocab_size] 的 logits 向量；然后：

1. softmax → 概率分布；
2. sample (带 temperature、top-k、top-p) → 选一个 token id；
3. 通过 tokenizer 反查 (BPE / SentencePiece) → UTF-8 字符串；
4. 写入 Server-Sent Events (SSE) 流；
5. 经过 L7 proxy → CDN edge → 你的浏览器；
6. 浏览器把字节追加到 DOM 文本节点——你看到了那个字。

整个过程对一次“对话回复”会重复几百次，每个 token 30-100 毫秒——你看到的“流式输出”就是这样产生的。

KV cache 的关键作用

朴素实现每个 token 都重新计算前面所有 token 的 attention——是 $O(T^2)$ 的代价。**KV cache** (键值缓存) 把每层的 K、V 张量留在显存里，下一个 token 只算新增的那一行——把每 token 复杂度降到 $O(T)$ 。这是 vLLM、SGLang 等推理引擎的核心优化，也是讲 14-17 并发数据结构在 AI serving 里的真实落地：缓存命中 / 失效 / 驱逐策略 = 经典内核 cache 问题。

3.5 本章小结

LLM 推理在硬件上 = 巨大的 matmul + 巨大的 KV cache。matmul 是 SIMT 的 killer app——三重循环天然映射到 GPU thread grid；KV cache 是并发数据结构的现代化身——分页、驱逐、共享。你按下回车到看到第一个 token 之间的所有故事，都是本课程前 20 讲讲过的原理在工业 AI 场景下的具体化。

4 总结与延伸

4.1 讲者核心观点

1. 操作系统课的所有知识在数据中心 + AI 推理场景里全部”激活”——这门课不是为了考试，是为了让你能看懂任何一个工业级系统。
2. 并发部分的五讲（13-17）+ scale 三讲（18-20）+ 本讲串成一个故事：从 spawn/join 到 <<<grid, block>>>, 一以贯之。
3. C10K → C10M → C10B：每一次数量级跃升都催生新一代并发原语（epoll → goroutine → 分布式 service mesh）。
4. LLM 推理在原理层面并不神秘——读懂 llm.c 的人都能理解每一层 transformer 在做什么。神秘的是工程规模。
5. Tensor 是数组的广义版；attention 的三重循环天生适合 SIMT；KV cache 是经典 cache 问题在新场景的复刻。
6. 数据中心是当代信息文明的”幕后英雄”——你和它的接口就是一个 IP 地址、几百毫秒的网络延迟，但背后是整个产业链 + 整个学科的支撑。

4.2 自测三问

看完本讲，能答上这三题吗？

1. 当你在浏览器里输入 api.deepseek.com，到收到第一个 token 之间至少跨越了多少个进程 / 线程 / GPU kernel？挑一个环节展开讲讲它和操作系统课哪一讲对应。
2. 读 karpathy/llm.c 的 matmul_forward_kernel (CUDA 版)；用本课讲过的概念解释每一行 <<<grid, block>>> 在干嘛。
3. KV cache 在多用户共享时怎么实现高效查找 / 驱逐？这和讲 17 (并发 bugs) / 讲 18 (并发数据结构) 哪些概念相通？

4.3 与课程其他部分的 synthesis

- 与讲 5-7 (进程 / mmap / fd)：数据中心后端服务的 socket fd、内存映射文件、零拷贝路径——这些 API 在 C10M 场景下被压榨到极限。
- 与讲 12 (Linux from scratch)：每一台 GPU 服务器都是从 initramfs 启动起来的；CUDA driver 也是一个 init 脚本里 modprobe 上的内核模块。
- 与讲 13-17 (并发系列)：mutex / 条件变量 / 信号量 / 并发数据结构、并发 bugs——这些原语真实地构成了 vLLM / SGLang 等推理引擎的调度核心。
- 与讲 18 (并行算法) + 讲 19 (协程) + 讲 20 (SIMT/GPU)：这三讲拼图直接构成了 LLM serving 后端的并发栈：上层用协程接受请求，中层用 work-stealing 调度任务，下层用 CUDA kernel 跑 matmul。

4.4 拓展阅读

- Andrej Karpathy, *llm.c*: <https://github.com/karpathy/llm.c> ——2000 行 C 看懂 GPT-2 训练与推理.
- Andrej Karpathy, *Let's reproduce GPT-2 (124M)* (YouTube 视频, 4 小时手把手) .
- Dan Kegel, *The C10K Problem*: <http://www.kegel.com/c10k.html>.
- Vaswani *et al.*, *Attention is All You Need*, NeurIPS 2017.
- vLLM 项目: <https://github.com/vllm-project/vllm> ——现代 LLM serving 引擎, KV cache + PagedAttention 的工业级实现.
- SGLang 项目: <https://github.com/sgl-project/sglang> ——北大团队的高性能推理框架.
- Krizhevsky *et al.*, *ImageNet Classification with Deep Convolutional Neural Networks*, NIPS 2012 (AlexNet 原始论文) .
- CISCO, *What Is a Data Center?* (数据中心入门白皮书) .

4.5 致谢

本讲是 jyy 2026 春季《操作系统原理》并发部分的收官——也是 lecture-to-notes 项目目前覆盖的最后一讲。感谢蒋炎岩老师把整门课”非常长的并发部分”用一个端到端的故事收得这么漂亮——每一节课、每一个 API、每一行 `<<<grid, block>>>`, 最终都连成了你按下回车后那 1 秒钟里的全部。